

编 号: _
审定成绩: _____

重庆邮电大学

本科毕业设计（论文）



中文题目	基于移动平台的
	固定目标识别及瞄准系统设计
英文题目	Fixed-Target Recognition and
	Aiming System on a Mobile Platform
学院名称	自动化/工业物联网学院
学生姓名	杨应添
专 业	自动化
班 级	08902201
学 号	2022213347
指导教师	蔡军教授
答辩人	姓名职称

2026 年 6 月

重庆邮电大学教务处制

学院本科毕业设计(论文)诚信承诺书

本人郑重承诺：

我向学院呈交的论文《 》，是本人在指导教师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明并致谢。本人完全意识到本声明的法律结果由本人承担。

年级

专业

班级

承诺人签名

年 月 日

学位论文版权使用授权书

本人完全了解重庆邮电大学有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权重庆邮电大学可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

(注：保密的学位论文在解密后适用本授权书。)

学生签名:

指导老师签名:

日期: 年 月 日

日期: 年 月 日

摘要

随着无人化装备向低成本、智能化方向持续演进，在移动平台上实现固定目标的实时识别与精确瞄准已成为领域内的重要研究方向。现有方案普遍面临成本高昂、部署复杂或实时性不足等瓶颈。针对上述问题，本文设计并实现了一种基于移动平台的固定目标识别与瞄准系统，以“千元级成本、纯视觉+IMU 紧耦合”为设计约束，提出了一套具备工程可复现性的低成本实现方案。

系统采用三节点分布式异构架构。Linux 视觉节点负责目标检测、PnP 位姿解算及跟踪状态管理；STM32H743 云台主控节点以 480 MHz 主频运行 RT-Thread 实时操作系统，执行双闭环姿态控制与 IMU 前馈补偿；MSPM0G3507 底盘主控节点独立完成八路灰度传感器循迹与差速编码器 PI 速度闭环。各节点通过串口协议进行数据交互，彼此独立运行、互不干扰。

在视觉识别与跟踪方面，构建了 HSV 颜色阈值分割、轮廓几何约束与 PnP 重投影尺寸校验的三级筛选流程，可对 A4 白纸目标实现可靠的检测与中心定位。结合三线程与双有界队列的并发架构，视觉处理吞吐量稳定维持在 25 FPS 以上。

云台控制采用双闭环结构：外层为像素误差驱动的 APID 控制器，内层为 IMU 角速度与加速度前馈补偿项，其中前馈环节的设计借鉴了自抗扰控制思想。在平台突加速或受振动冲击时，该结构可有效降低相位滞后。实测结果表明，瞄准线峰值偏差降低 30% 以上，稳态指向误差满足 $\leq 0.8 \text{ mrad}$ 的设计指标。IMU 选用 HiPNUC 六轴惯性传感器，以 100 Hz 输出角速度与姿态信息，通过 hipnuc_dec 协议模块完成字节流解析与 CRC 校验。

底盘控制采用差速运动学模型进行速度分解，左右轮各自执行增量式 PI 调速与 AB 相编码器反馈闭环。循迹策略融合归一化灰度加权偏差估计与一阶滞后低通滤波，有效抑制了传感噪声引起的控制抖动。

实验结果表明，室内标准光照条件下视觉帧率稳定超过 25 FPS，IMU 采样保持在 100 Hz，底盘可稳定完成多圈循迹。整机器件成本控制在千元以内，验证了所述方案的工程可行性与实用价值。

关键词：视觉伺服，目标识别，云台控制，IMU 前馈，移动机器人

Abstract

With the growing demand for low-cost intelligent unmanned systems, real-time recognition and accurate aiming at fixed targets on mobile platforms has become a prominent research topic. Existing solutions are typically hindered by high cost, deployment complexity, or insufficient real-time performance. This thesis proposes and implements a fixed-target recognition and aiming system for mobile platforms, designed under the constraints of less than 1000 RMB and “pure vision + IMU tight coupling,” providing an engineering-practical low-cost solution.

The system adopts a three-node distributed heterogeneous architecture. The Linux vision node handles target detection, PnP pose estimation, and tracking state management. The STM32H743 gimbal controller, running the RT-Thread real-time operating system at 480 MHz, executes a dual-loop attitude control algorithm with IMU feedforward compensation. The MSPM0G3507 chassis controller independently manages eight-sensor grayscale line tracking and differential-drive encoder PI speed control. The three nodes communicate via serial protocols and operate without mutual interference.

For visual detection and tracking, a three-stage filtering strategy is adopted: HSV color threshold segmentation, contour geometric constraint verification, and PnP reprojection size validation. This enables robust detection and center localization of A4-paper targets. A three-thread plus dual-bounded queue architecture ensures stable visual throughput above 25 FPS.

For gimbal control, a dual-loop architecture is proposed: an outer APID loop driven by pixel error, and an inner feedforward loop driven by IMU angular velocity and acceleration. The feedforward design is inspired by Active Disturbance Rejection Control (ADRC). Under platform acceleration or vibration disturbances, the architecture significantly reduces phase lag, achieving a reduction of more than 30% reduction in peak aiming-line error in experiments. The steady-state pointing error satisfies the design target of ≤ 0.8 mrad. The HiPNUC six-axis IMU provides 100 Hz angular velocity and attitude output, decoded via the `hipnuc_dec` protocol module.

For chassis control, a differential kinematics model decomposes forward and angular velocity commands into individual wheel speed targets. Each wheel employs an incremental PI speed control with AB-phase encoder feedback. The line-following strategy combines normalized grayscale weighted deviation estimation with a first-order low-pass filter to suppress control jitter.

Experimental results demonstrate that the visual frame rate stably exceeds 25 FPS under standard indoor lighting, the IMU maintains 100 Hz sampling, and the chassis completes multi-lap line-following tasks reliably. The total hardware cost remains within the budget, validating the feasibility and engineering value of the proposed system.

KEY WORDS: Visual Servo, Target Recognition, Gimbal Control, IMU Feedforward, Mobile Robot

目录

第 1 章	引言	1
1.1	研究背景	1
1.1.1	应用背景	1
1.1.2	技术背景	1
1.2	研究目的及意义	2
1.2.1	研究目的	2
1.2.2	研究意义	2
1.3	研究现状及设计思路	2
1.3.1	国外研究现状	2
1.3.2	国内研究现状	3
1.3.3	存在的主要问题	3
1.3.4	本文的设计思路	4
1.4	发展趋势	4
1.5	主要内容和论文组织	4
第 2 章	系统总体设计	5
2.1	系统设计工作原理及原理框图	5
2.1.1	基本工作原理	5
2.1.2	系统原理框图与模块说明	5
2.2	系统设计的要求	5
2.2.1	功能性要求	5
2.2.2	性能指标要求	6
2.2.3	约束条件与设计边界	6
2.3	关键器件选型与接口设计	6
2.3.1	主控器件	7
2.3.2	传感器与视觉模块	7
2.3.3	执行机构与接口设计	7
2.4	本章小结	8
第 3 章	系统硬件设计	9
3.1	单片机控制核心电路	9
3.2	IMU 数据采集电路	10
3.2.1	IMU 内部结构与工作模式	10
3.2.2	IMU 时序与采样配置	10
3.2.3	IMU 接口连接与抗干扰设计	10
3.3	图像采集与通信电路	10
3.3.1	USB 摄像头接入原理	10
3.3.2	图像数据链路设计	11
3.4	云台驱动与电机控制电路	11
3.4.1	电机驱动原理	11
3.4.2	RS485 总线接口与电源设计	11
3.5	自动循迹底盘控制电路	12

3.6	本章小结	14
第 4 章	系统软件设计	15
4.1	软件总体架构与流程	15
4.1.1	程序分层与任务划分	15
4.1.2	软件主流程图	15
4.2	IMU 数据处理程序	16
4.2.1	数据采集与滤波	16
4.2.2	姿态解算与角速度发布	16
4.3	视觉识别与跟踪程序	17
4.3.1	视觉线程架构与数据流	17
4.3.2	目标检测与几何筛选流程	18
4.3.3	PnP 位姿解算与目标量生成	19
4.3.4	丢失重捕获与串口通信协议	19
4.4	双闭环控制程序设计	20
4.4.1	像素误差外环 PID	20
4.4.2	IMU 角速度前馈内环	20
4.5	通信与上位机控制程序	21
4.5.1	USB-CDC 通信协议	21
4.5.2	Linux 端控制界面与参数下发	21
4.6	系统主控程序设计与异常处理	22
4.7	差速底盘工程的软件实现	22
4.7.1	控制主循环与运行模式	22
4.7.2	红外循迹与路口判别策略	23
4.7.3	固定圈速实现与圈数管理	23
4.7.4	差速运动学与轮速闭环	23
4.7.5	作为系统子模块的协同与可靠性优势	24
4.8	本章小结	24
第 5 章	系统调试与结果分析	27
5.1	调试环境与测试方案	27
5.2	图像识别与跟踪效果测试	27
5.3	姿态采集与稳定性测试	28
5.4	云台控制与稳态误差测试	28
5.5	本章小结	28
第 6 章	总结与展望	29
6.1	主要工作与创新点	29
6.2	不足与改进方向	29
6.3	后续研究工作展望	29
参考文献		33
致谢		35
附录 A	科技写作中非学术型低级错误的主要表现	36
附录 B	英文翻译	37

第 1 章 引言

1.1 研究背景

1.1.1 应用背景

近三十年来，无人化装备从“遥控式”逐步向具备更高自主能力的“自主式”平台演进。在此背景下，移动平台上固定目标的高精度识别与瞄准已成为工程与研究的核心课题之一。早期军事需求是推动该领域发展的重要动力：20 世纪 90 年代，美军在“猎人”无人机平台上开展光电瞄准系统试验，开创了以视觉测量为核心的“平台运动—目标静止”瞄准范式。2010 年后，北约及主流军工企业将“视觉伺服 + 惯性稳定”确立为移动平台火控系统的重要设计原则，该方向由此从实验室验证迈向工程实用阶段^[1,2]。

除国防领域外，民用与商用场景对轻量化、低成本、易部署的移动瞄准与跟踪方案同样提出了迫切需求。安防监控、交通监测、无人机巡检、精细农业（如病虫害自动定位喷洒）及影视拍摄等应用，均需在受限的功耗与重量条件下实现足够的指向精度与鲁棒跟踪能力。以城乡安防为例，移动巡检平台需在多变光照与部分遮挡条件下持续评估目标位置并保持指向；农业施药场景中，设备成本与续航能力是硬性约束，大功耗传感器阵列难以适用，轻量级视觉-惯导方案遂成为首选。

从工程实现角度看，此类系统面临一系列固有约束：传感器与处理单元须在有限的尺寸、重量与功耗预算内协调工作；时间同步精度与数据链路延迟直接影响闭环控制性能；复杂环境下还需保证对遮挡、光照变化及多目标干扰的鲁棒性。正是这些现实需求，推动近年来的研究从单纯追求检测精度，转向更注重系统级时延、能耗与可复现性的综合优化，为面向教学与工程验证的低成本实现提供了可行的研究空间。

1.1.2 技术背景

移动平台瞄准系统的实现依赖于视觉传感、惯性测量、实时嵌入式计算与机电稳定（云台）等多学科的交叉与协同。国内在该领域起步较晚，但自 2015 年以来已有若干型号的车载光电转台投入使用，表明在小型化陀螺稳定与实时图像跟踪方面已取得工程级突破。目前衡量轻量化系统可行性的常用指标包括“千元级成本、5 W 功耗、1000 g 重量”等约束条件，这些限制正是推动学术研究向工程实现靠拢的重要动因。

关键技术维度可归纳为四个方面。其一为传感器选型：从工业级相机、激光雷达到民用 USB 摄像头、低成本六轴 IMU，选择结果直接决定了系统在精度、体积、功耗与成本之间的权衡关系。其二为实时处理与算法实现：传统方法（如 HOG+SVM）在资源受限环境下具备良好的性价比，而轻量化神经网络（如 MobileNet、Tiny-YOLO）近年来通过模型压缩与量化技术，已逐步能够在嵌入式平台上达到可用帧率。近期面向微控制器与嵌入式平台的轻量化检测与跟踪工作（例如 TinyissimoYOLO、DSORT-MCU、LeYOLO 等）证明了在受限内存与算力下实现实时检测的可行性，并成为工程实现的重要参考^[3-5]。同时，面向高效特征描述与多目标实时跟踪的新方法（如 EdgePoint2、TCBTrack）为嵌入式跟踪系统提升效率与鲁棒性提供了可借鉴的方法^[6,7]。其三为时间同步与时序保障：高频 IMU 与低频视觉数据的紧耦合要求精确的时间戳与低延迟通信，否则前馈补偿与闭环控制效能将显著下降。其四为机械与控制设计：云台机械结构的合理性、减震方案以及电机驱动策略，均对最终指向精度产生直接影响。

在软件架构方面，嵌入式平台（如 STM32H7）与上位机（Linux 视觉端）的分工协作模式已被广泛采用。上位机承担复杂的视觉推理与图像处理任务，主控板负责 IMU 采样、闭环控制与实时执行，在保证性能的同时避免了主控端算力过载。常见的工程实

践还包括：利用 DMA 与中断机制确保 IMU 数据的高精度采样、通过 RS485 等差分总线实现云台电机的可靠通信，以及在控制环路中引入前馈与预测补偿来降低视觉反馈滞后带来的性能损失。

总体来看，技术背景呈现两条并行的发展脉络：一方面，追求极致精度的方案仍需依赖高端传感器与大算力平台；另一方面，为满足成本与功耗约束，研究重心正逐渐转向算法与系统的协同优化——包括轻量模型设计、传感器融合与时序对齐等方向。近期实时检测模型在“开放词汇与航拍场景”的扩展，也进一步说明了实时框架在复杂任务下的可迁移性与工程潜力^[8,9]。后者为本科与工程验证级别的研究提供了可行且具备实践意义的实现路径。

1.2 研究目的及意义

1.2.1 研究目的

本课题设计并实现基于移动平台的固定目标识别及瞄准系统，采用“纯视觉+IMU”紧耦合的轻量化方案，在满足成本约束的前提下，实现以下具体目标：

1. 构建硬件平台与通信基础：搭建以 STM32H7 为核心的小型化云台系统，集成 USB 摄像头与六轴 IMU，实现云台的实时远程控制；
2. 实现环境感知与角速度采集：完成高频（ $\geq 100\text{ Hz}$ ）运动基座状态信息采集与处理；
3. 完成视觉跟踪与像素闭环：实现 25 FPS 连续跟踪，瞄准线稳态误差控制在 0.8 mrad 以内；
4. 集成 IMU 前馈与振动抑制：在像素闭环基础上引入 IMU 补偿，对 $10\sim 200\text{ Hz}$ 频段进行实时补偿。

1.2.2 研究意义

本研究在学术层面填补了国内轻量化动基座瞄准方案的部分空白，在应用层面则为无人系统提供了一种可复现、低成本的参考路径。通过构建“纯视觉+低成本”的设计框架，为未来接入激光雷达或事件相机等传感器预留了硬件与算法接口，具有良好的可扩展性。

1.3 研究现状及设计思路

1.3.1 国外研究现状

国外在该领域起步较早、投入较大，普遍遵循“高精度、高成本、重平台”的技术路线。在多传感器融合策略上，激光雷达、视觉与惯导的三元融合架构已成为主流方案。

以美国卡内基梅隆大学 2022 年的公开实验为例，其无人地面车辆搭载了 128 线激光雷达、工业级可见光相机（ $1920\times 1440@30\text{ Hz}$ ）及光纤陀螺，构成三位一体的传感器系统。该平台通过 Transformer 检测头^[7]实现了 30 Hz 的实时目标识别，能够在复杂城市环境中精确定位固定目标，像素误差控制在 2 pix 以内，折合角度精度约 0.3 mrad 。然而，整套系统成本超过 2 万美元，功耗高达 45 W ，对移动平台的续航能力构成显著制约。此类方案难以满足本科教学层面对“低成本、易复现”的基本要求。

德国弗劳恩霍夫应用研究所（Fraunhofer IAO）进一步将 FPGA+GPU 异构计算架构引入瞄准闭环控制系统。其方案在 GPU 上运行 Faster R-CNN 目标检测算法，同时在 FPGA 上并行执行 IMU 前馈补偿，利用异构计算的高吞吐特性将定位误差压缩至 1 cm 量级（对应约 0.05 mrad 的角误差）。然而，该系统重量达 6 kg ，需 24 V 、 10 A 供电，单轴陀螺稳定分系统成本即超过 5000 元，难以在 5 kg 以下的微型无人车上部署。

在传感器层面,国外普遍采用工业级高精度 IMU (漂移率 $< 1^\circ/\text{h}$) 与光纤陀螺 (精度 $< 0.01^\circ/\text{s}$), 其成本较民用级传感器高出逾一个数量级。Xsens 公司的 MTi-630 AHRS 惯性测量单元价格在 8000 元以上, 光纤陀螺类产品更是数万元起步。这种以传感器堆砌换取高精度的策略, 在成本与功耗层面代价过高。

综上所述, 国外研究强调“多传感器融合 + 大算力 + 高精度”的技术路线, 通过物理层的传感器冗余与算法层的计算冗余实现极致性能。尽管精度指标上遥遥领先^[2,10], 该技术路线对微型无人车、教学实验平台等资源受限场景存在天然的适配性问题。

1.3.2 国内研究现状

受成本约束限制, 国内研究普遍采取了更为务实的“纯视觉 + 惯导”轻量化路线^[11]。其核心理念在于通过精细化的算法设计弥补硬件配置的不足, 在有限的计算资源与传感器精度约束下, 实现工程可行的瞄准精度, 这需要采用模型压缩、量化等优化技术^[12]。

早期探索方面, 一些工作在低分辨率、低算力平台上采用传统特征方法实现了可用的检测效果 (例如在低分辨率嵌入式平台上的实现报告), 验证了传统机器学习方法在资源受限场景下的可行性。然而, 该研究未能有效解决动基座-静止目标打击模式中平台运动引发的振动零偏问题——该零偏通常在 100 ms 量级, 足以破坏瞄准精度, 成为后续研究的关键课题^[11]。

进入深度学习时代, 齐鲁工业大学 2021 年的学位论文取得了重要进展。该研究将 Faster R-CNN^[1] 的骨干网络替换为轻量化 MobileNet^[2], 在 STM32 微控制器与 OpenMV 嵌入式视觉模组的架构上实现了 20 FPS 的实时检测, mAP 达 72%, 较 HOG+SVM 方案有明显提升。随后 MobileNetV2^[13] 进一步改进了网络结构以减少计算量, 为嵌入式平台上的高效推理奠定了基础。但该工作同时暴露了一个突出的工程问题: 在遮挡场景或强光突变条件下, 目标丢失率仍高达 6% 8%, 系统鲁棒性不足。

电子科技大学 2022 年的一项嵌入式视觉研究进一步量化了这一瓶颈。性能分析表明, 即便采用轻量化的 MobileNet CNN 架构, 在 640×480 的输入分辨率下, 推理过程仍会占用嵌入式 GPU 约 80% 的算力资源。这种高负载状态不仅导致瞄准延迟超过 100 ms (远超 10 ms 量级的实时性要求), 还会引发显著的功耗上升与热量积累, 对嵌入式平台的稳定运行造成威胁。

在算法层面, 国内在 IMU 数据处理方面也取得了一定进展。多个研究团队对 IMU 零偏校准与温漂补偿进行了深入探索, 但多数工作仍停留在离线标定阶段, 实时的自适应标定与在线零偏补偿技术尚不多见。此外, “像素误差 + 角速度前馈”紧耦合双闭环控制架构在国内仍处于萌芽阶段, 尚缺乏系统的理论分析与工程验证。

总体而言, 国内轻量化方案在振动抑制、实时性与鲁棒性方面与国外存在约 1 至 2 年的工程化差距。这一差距的成因可归纳为三点: 其一, 国内团队在算法创新方面积累不足, 倾向于跟踪国外最新论文而忽视工程适配性; 其二, 轻量化 IMU 与高性能嵌入式芯片在国内的可获得性有限; 其三, “纯视觉 + IMU”紧耦合完整系统的设计与验证缺乏学科交叉的系统性研究。

1.3.3 存在的主要问题

综合国内外研究现状, 当前存在的主要问题包括:

1. 国外方案在成本、体积和功耗方面存在极端不平衡, 不适用于轻量化应用;
2. 国内轻量化方案在振动抑制、实时性与鲁棒性上与国外存在 1~2 年的工程化差距;
3. “像素-角速度”一体化设计尚处萌芽阶段, “纯视觉 + IMU”紧耦合的轻量化方案缺乏系统研究;
4. 移动平台动态运动对瞄准精度的影响补偿不足, 需要建立双闭环控制架构。

1.3.4 本文的设计思路

基于上述问题分析，本文确立了以下设计原则：

1. 架构选择：采用“纯视觉+IMU”紧耦合方案，摒弃激光雷达等昂贵传感器，在“千元级成本、5W 功耗”约束下实现可接受的瞄准精度；
2. 特征算法：选用 HOG、ORB 等传统特征与模板匹配方法，无需 GPU 支持即可在嵌入式平台上运行，确保帧率超过 25 FPS；
3. 控制架构：构建“像素误差外环+角速度前馈内环”的双闭环结构，利用 IMU 数据提前预估图像偏移量，有效降低平台机动引入的瞄准误差；
4. 时间同步：在硬件层面实现图像与 IMU 数据时间戳的精确对齐，为紧耦合控制奠定基础。

上述设计在保证工程可复现性的同时，为未来的技术升级预留了扩展空间。

1.4 发展趋势

国内移动平台瞄准系统正从“多传感器融合”向“视觉-IMU”紧耦合的轻量化方向转型。硬件层面，国产微控制器主频已突破 480 MHz，内置 FPU 与 DSP 指令集，为“视觉跟踪+IMU 前馈”的同频运行提供了算力支撑。算法层面，HOG、ORB 等传统特征方法无需 GPU 即可达到较高帧率，且采样频率与 IMU 接近，便于时间戳对齐。控制层面，“像素误差外环、角速度前馈内环”的双闭环架构已趋成熟，可将平台机动引起的瞄准线误差压缩 30% 以上。

综上所述，“纯视觉+IMU”紧耦合已成为低成本动基座瞄准领域的一个明确发展趋势，为本科毕业设计提供了可落地、可复现的技术路线窗口。

1.5 主要内容和论文组织

本文针对移动平台的固定目标识别与瞄准问题，设计并实现了一套低成本系统。全文共分为 6 章，各章内容组织如下：

第 1 章 阐述课题的研究背景与应用意义，综述国内外研究现状、存在的问题、设计思路及发展趋势，论证本课题研究的必要性及可行性。

第 2 章 给出系统整体方案，明确“Linux 视觉端+云台主控+底盘主控”的三节点分布式架构，梳理各节点间的通信协议与数据流，阐明系统的工作原理与总体框图，并给出功能需求与性能指标约束。

第 3 章 详述各硬件模块的设计方案，包括 STM32H7 主控核心电路、HiPNUC 六轴 IMU 采集电路、USB 摄像头接入方式、MS4010 双轴云台驱动电路及底盘传感器接口，最后给出系统整机连接关系图。

第 4 章 详细描述软件的分层架构与实现细节，涵盖 IMU 角速度采集与处理、视觉目标检测与跟踪、像素误差外环与 IMU 前馈内环的双闭环控制算法、底盘差速驱动与八路灰度循迹控制，以及节点间通信协议的实现。

第 5 章 对系统进行分模块与整机调试，包括图像识别与跟踪效果测试、IMU 采样稳定性与补偿性能验证、云台闭环稳态误差测试及底盘循迹精度分析，给出各项性能指标的实测结果与分析。

第 6 章 归纳本文的主要工作与创新点，分析系统存在的不足，并指出后续可深化的研究方向与技术升级路径。

第2章 系统总体设计

本章围绕“基于移动平台的固定目标识别及瞄准系统”阐述总体设计方案，重点说明系统工作原理、模块间关系及性能约束条件，为后续的器件选型、硬件实现与软件设计提供统一的依据。

2.1 系统设计工作原理及原理框图

2.1.1 基本工作原理

系统以“视觉感知+惯性测量+双闭环控制”为核心链路。摄像头负责目标图像采集与误差提取，IMU提供高频角速度信息，控制器依据像素偏差与角速度前馈生成云台控制量，实现对固定目标的持续指向与误差抑制。

在运行过程中，系统遵循“采集-估计-控制-反馈”的循环流程。外环以像素误差为主要控制对象，内环以角速度补偿为核心任务，二者协同以兼顾稳态精度与动态响应性能。从计算节点维度划分，系统由三个协同工作的处理单元构成：

(1) **Linux 视觉节点**：基于通用计算平台运行 OpenCV 视觉管线，完成目标检测、位姿解算与跟踪状态管理，结果通过串口协议下发至云台主控；

(2) **云台主控节点 (STM32H7)**：以 480 MHz 主频实时处理 IMU 数据、执行双轴伺服控制与瞄准解算，通过 RS485 总线驱动 MS4010 双轴电机；

(3) **底盘主控节点 (独立单片机)**：独立执行八路灰度传感器循迹、编码器 PI 速度闭环与圈次任务，与云台节点仅在接口层面进行协同，控制上互不耦合，互不干扰。

该三节点架构将视觉计算、姿态控制与底盘机动分别部署于各自最适宜的平台，在降低单节点负担的同时提升了系统的可扩展性。

2.1.2 系统原理框图与模块说明

系统总体由移动底盘、云台执行机构、视觉采集模块、IMU 模块、主控与通信模块、上位机监控模块构成。各模块间通过标准接口进行数据交互，形成可扩展、可维护的分层结构。系统总体框图如图2.1所示。

如图所示，系统可划分为三个功能子系统：瞄准模块（STM32H7 云台控制 MCU、陀螺仪与云台电机）、视觉模块（摄像头与树莓派/Linux 视觉节点）以及自动循迹底盘（底盘控制 MCU、八路灰度传感器与底盘电机）。各子系统通过串口或 RS485 总线互联，由 12 V 电源总线统一供电。

主控侧负责实时控制与状态发布，上位机侧负责参数配置、可视化监测与实验数据记录。该分层架构便于后续算法替换与功能扩展。

在节点间通信层面，各链路的数据类型与协议存在差异：Linux 视觉节点采用固定帧格式通过串口向云台主控发送目标像素坐标与置信标志；云台主控通过 RS485 总线与 MS4010 电机通信，完成角度指令打包、响应解析与超时检测；底盘主控作为独立控制器运行内部闭环，与上层通过预定义接口进行模式同步。这种“链路分离”的设计使各子系统在对方降级时仍能维持各自的最小功能闭环，从而提升了整体系统的鲁棒性。

2.2 系统设计要求

2.2.1 功能性要求

系统应具备目标识别与连续跟踪、云台姿态控制、实时状态回传、参数在线整定等基本功能。其核心任务是在移动平台存在运动扰动的前提下维持瞄准线的稳定。

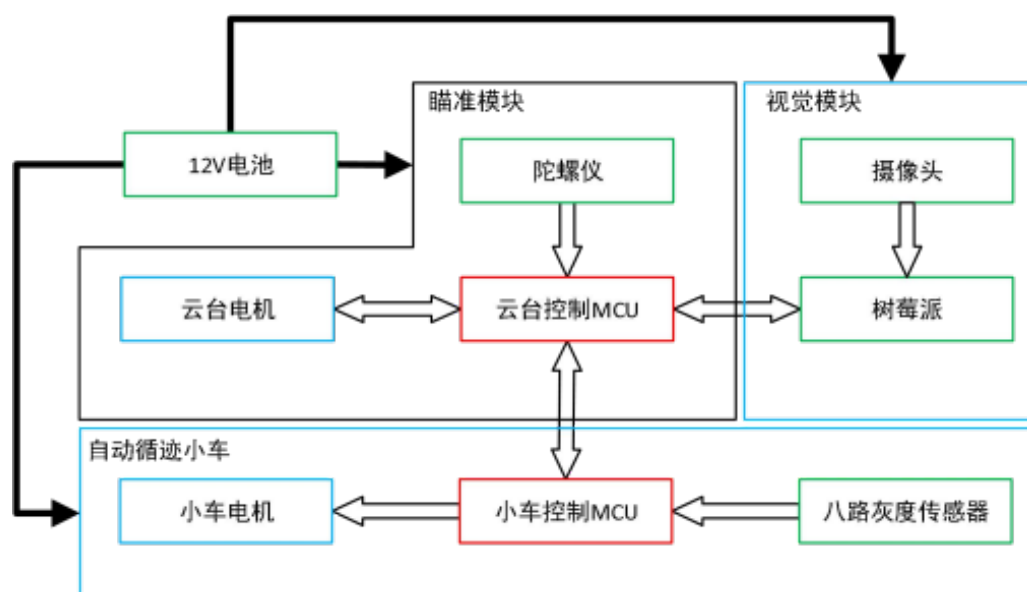


图 2.1 系统总体框图

2.2.2 性能指标要求

结合开题目标，系统设计以低成本平台为约束，重点满足以下定量指标，如表2.1所示。

指标类别	指标项	设计目标	说明
视觉实时性	视觉帧率	$\geq 25 \text{ FPS}$	室内标准光照场景
惯性采样	IMU 采样频率	$\geq 100 \text{ Hz}$	角速度前馈需求
瞄准精度	稳态指向误差	$\leq 0.8 \text{ mrad}$	静态目标双闭环收敛后
成本约束	整机器件成本	千元级 ($\leq \text{¥}1000$)	不含通用计算平台
功耗约束	系统功耗	$\leq 5 \text{ W}$	嵌入式部分
重量约束	嵌入式系统重量	$\leq 1000 \text{ g}$	含云台机构

表 2.1 系统性能指标要求

上述指标均基于具体工程需求确定。 0.8 mrad 的稳态误差指标源于对典型室内应用场景的分析与估算； 25 FPS 的视觉帧率确保像素闭环具备充足的带宽裕量； 100 Hz 的 IMU 采样频率则保证前馈补偿能够对平台机动做出及时响应。

2.2.3 约束条件与设计边界

系统设计受限于本科毕设条件与嵌入式平台算力，遵循“低成本、低功耗、可复现”的工程边界，不依赖高功耗 GPU 与昂贵传感器。在此约束框架下，优先选择技术成熟、易于调试、便于复现的实现路线。

2.3 关键器件选型与接口设计

为落实总体设计目标，本节在“可获取性、接口兼容性、实时性、工程可复现性”等原则指导下，完成关键器件的选型与接口关系定义，使系统方案从架构层进一步落实到可实现的工程层面。

2.3.1 主控制器件

云台控制系统需同时处理来自 IMU 与视觉模块的多源数据，并在实时控制周期内完成角速度前馈补偿与伺服控制计算。因此，主控制器件须具备较高的主频、充足的存储资源及完善的外设接口，同时需支持定时控制、中断管理与通信协议扩展，并具有健全的开发生态以支持快速原型验证。

基于上述需求，本课题选用 STM32H7 系列微控制器作为云台主控平台。所选型号 STM32H743XIH6 的 CPU 主频高达 480 MHz，内置单精度与双精度浮点运算单元 (FPU) 及 DSP 指令集，可胜任 IMU 数据处理、控制律计算与通信管理等高实时性任务。STM32H7 平台生态成熟，支持移植 RT-Thread 实时操作系统，驱动资源丰富，有利于提高开发效率。相较其他微控制器方案（如 MSPM0G3507 主频仅 80 MHz），STM32H7 在计算能力与浮点性能上具有显著优势，能够更好地满足云台双轴伺服、IMU 前馈补偿与视觉数据融合等需要同频运行的实时控制需求。

2.3.2 传感器与视觉模块

系统采用六轴惯性测量单元 (IMU) 用于姿态感知与角速度前馈，其输出数据为动态补偿提供了高频的输入信息，能够在平台运动状态下有效提升瞄准稳定性。经过零偏校准与滤波处理后，惯性传感器的测量精度可以进一步提高。选取 IMU 时需要确保角速度测量稳定、采样频率可灵活配置、通信接口可靠且抗干扰能力强，同时与主控平台在通信协议和电气特性上兼容。

在视觉模块方面，课题采用 USB 摄像头作为图像输入源。与专用视觉芯片模块（如 MaixCAM 等）相比，USB 摄像头的部署成本低、驱动生态完善，便于与上位机图像处理流程集成。更重要的是，USB 摄像头配合 Linux 平台能够直接运行 OpenCV 传统视觉算法，在本课题以 HSV 色彩分割和 PnP 位姿解算为核心的目标识别链中，帧率表现稳定且一致性好。此外，摄像头体积小、重量轻，易于架设在云台机构上随动，符合轻量化的系统设计目标。

2.3.3 执行机构与接口设计

云台的执行机构需要具备快速响应、精度可调、驱动稳定和维护简便等特点，并须与主控的输出信号相互匹配。在电机方案选择中，本课题对比了两种技术路线：其一为张大头自闭环步进电机云台，具有精度高、通讯简单的优点，但两轴步进电机组合后机械体积偏大，不利于整机的轻量化设计；其二为采用 RS485 接口的小型无刷云台机构，在满足控制精度要求的同时具备更为紧凑的体积和更轻的重量，与 STM32H7 的输出信号具有良好的兼容性。综合结构、性能等多方面因素，选择方案二作为执行机构。通过合理的驱动电路设计与闭环控制策略，可实现方位与俯仰通道的协调控制。

系统的信号连接采用 RS485 总线串口通讯方式。STM32H743 通过 UART 外设与 RS485 收发器相连，按照 MS4010 协议向偏航轴 (ID1) 和俯仰轴 (ID2) 的电机发送速度或位置指令帧。相比于 PWM 驱动方案，RS485 总线具有更强的抗干扰能力和多节点扩展潜力，通信速率可达 1 Mbit/s，足以满足 500 Hz 的控制周期实时要求。清晰的接口定义规范有利于后续系统联调复杂度的降低。

同时，本系统充分利用 STM32H7 的丰富外设资源：串口用于与 RS485 模块和通信设备通讯，定时器负责周期性的控制任务调度，PWM 模块驱动执行器，而 I2C 或 SPI 接口用于传感器数据采集。IMU 与主控通过标准数字接口完成数据交互，摄像头则与上位机侧构成独立的图像采集链路。两路数据流在控制层按时间顺序进行融合处理，保证控制计算的时间一致性和数据同步性。

2.4 本章小结

本章围绕系统总体设计展开，分析了基于视觉与惯性双闭环控制的工作原理，提出了 Linux 视觉节点、STM32H7 云台主控与底盘主控三节点分布式架构，给出定量性能指标与设计约束，完成关键器件的选型与接口定义。该方案兼顾实时性、低成本与工程可复现性，明确了各子系统的职责边界与通信协议，为后续硬件实现、器件验证与软件详细设计提供了完整的工程依据与接口规范。

本章在总体架构和器件选型基础上,给出各硬件子模块的电路设计思路与连接关系,明确系统从器件到整机的实现路径。

主控电路包含最小系统、电源管理、时钟与下载调试接口。设计中重点保证供电稳定、复位可靠与信号完整，以提高系统连续运行能力。

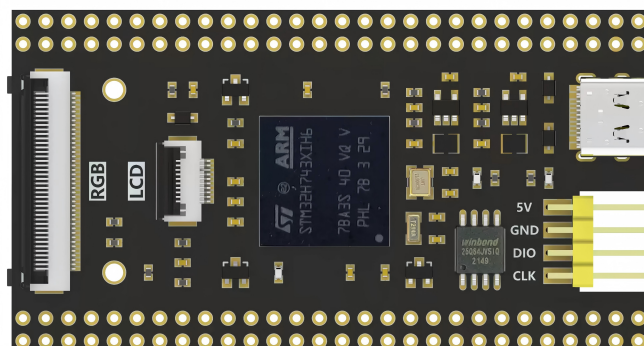


图 3.1 STM32H743 核心板实物图

3.2 IMU 数据采集电路

3.2.1 IMU 内部结构与工作模式

系统选用 HiPNUC 六轴 IMU 传感器，内置三轴加速度计与三轴陀螺仪，通过片内算法完成姿态解算，可直接输出角速度、姿态角与加速度等物理量。该传感器采用 UART 接口与主控通信，默认波特率 921600，输出频率可按需配置，典型工作模式下输出频率设置为 100 Hz 以匹配控制周期要求。

通过合理配置量程与滤波参数，可在兼顾动态响应与噪声抑制的同时，为后续前馈补偿提供高质量数据。

3.2.2 IMU 时序与采样配置

控制周期设置为 10 ms（对应采样频率 100 Hz），采用定时触发与 DMA 中断读取机制保证数据时序稳定。主控通过 hipnuc_dec 模块对 HiPNUC 协议进行字节流解码：先执行帧同步与 CRC 校验，再按数据包类型提取角速度 ω 、姿态角 θ 与加速度数据。解析成功后数据被发布到控制路径，供 IMU 前馈补偿计算使用。

对于角速度小幅波动，工程采用死区处理策略，避免将传感噪声直接放大到执行端，提升闭环控制品质。

3.2.3 IMU 接口连接与抗干扰设计

在接口连线中考虑去耦、电源滤波与布线隔离，降低电机驱动等强干扰源对惯性数据的影响，提升测量可靠性。

3.3 图像采集与通信电路

3.3.1 USB 摄像头接入原理

摄像头通过 USB 链路与上位机连接，形成图像采集与处理通道。该方案可降低主控侧图像处理压力，便于快速验证视觉算法。

为消除镜头几何畸变对 PnP 位姿解算的影响，需对摄像头进行标定。本文采用张正

友棋盘格柔性标定方法^[14]，经棋盘格标定实验，所用摄像头的内参矩阵如式(3.1)所示：

$$\mathbf{K} = \begin{bmatrix} 435.37 & 0 & 318.68 \\ 0 & 433.86 & 226.72 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.1)$$

其中， $f_x=435.37 \text{ px}$ 、 $f_y=433.86 \text{ px}$ 为焦距， $(c_x, c_y)=(318.68, 226.72) \text{ px}$ 为主点坐标。标定还同步获得径向畸变系数，标定后重投影误差可降至次像素级别，满足后续 A4 纸角点定位精度需求。

3.3.2 图像数据链路设计

图像链路设计关注带宽、延迟与稳定性，确保视频流在实验帧率下可持续传输，为实时跟踪与控制反馈提供数据支持。

3.4 云台驱动与电机控制电路

3.4.1 电机驱动原理

云台采用 MS4010 双通道电机驱动结构。偏航轴（Yaw）对应电机 ID 1，俯仰轴（Pitch）对应电机 ID 2，两轴通过 RS485 总线与主控通信，可实现位置模式与速度模式的在线切换，满足指向和跟踪两类控制需求。

系统上电后，主控扫描 RS485 总线发现 ID 1 与 ID 2 两台电机，完成云台初始化参数绑定、角度限位设置与回零动作，随后进入待命状态等待上层控制指令。MS4010 v3 电机实物如图3.2所示，机身侧面的 DIP 拨码开关用于设定 RS485 设备地址（即电机 ID），四位拨码对应二进制编码；底部印有 RS485 差分信号接线标识及电源正负极标注，安装时需严格核对极性以防驱动器损坏。

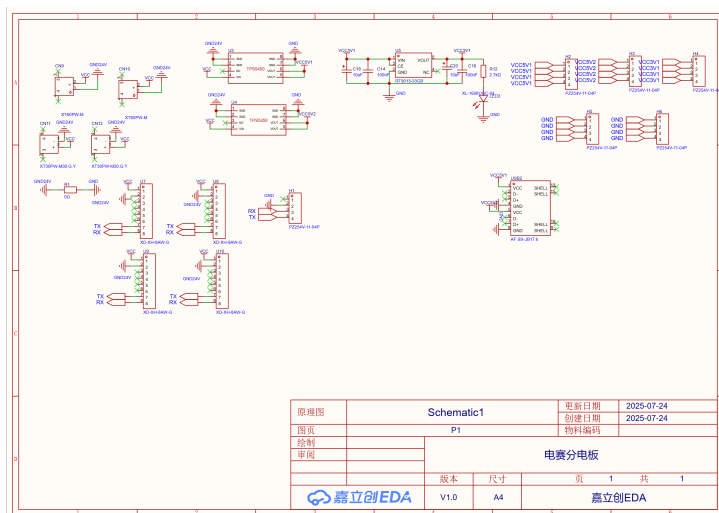


图 3.2 MS4010 v3 云台步进电机实物图

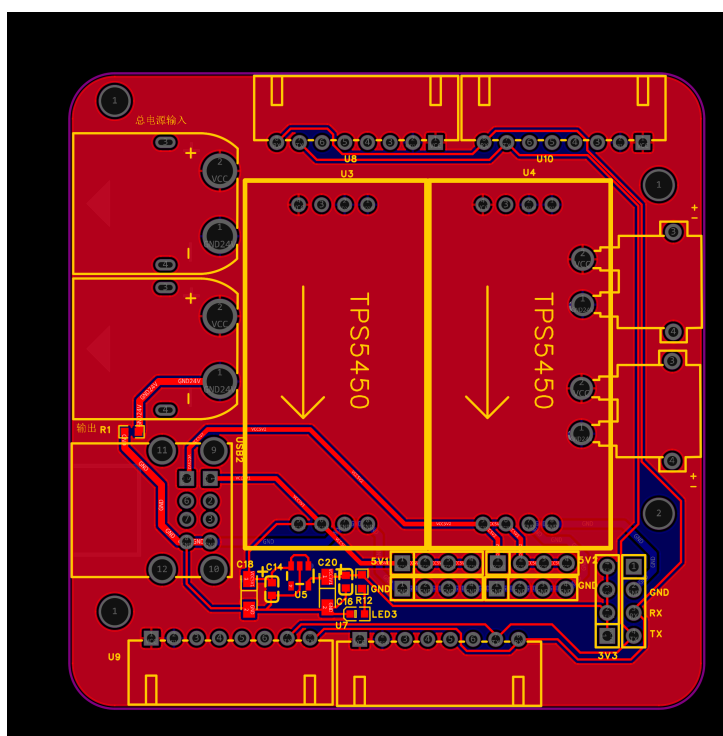
3.4.2 RS485 总线接口与电源设计

云台电机的控制信号经由 UART 外设驱动 RS485 收发器芯片后发出，MS4010 电机以差分信号接收指令，并将编码器状态通过同一总线回报。设计中对供电冗余、地线回流与过流保护进行约束，总线终端匹配电阻置于链路末端，消除长线反射，提升整机稳定性。

系统采用 12 V 锂电池统一供电，经 TPS5400 DC-DC 降压模块分别产生 5 V 与 3.3 V 轨为各子模块供电。电源电路原理图与 PCB 如图3.3(a)和图3.3(b)所示。



(a) 电源电路原理图



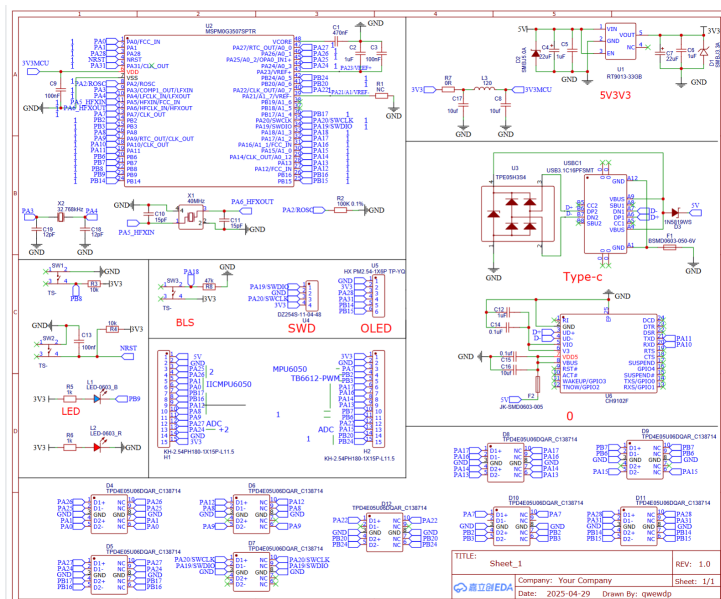
(b) 电源电路 PCB 布局

图 3.3 电源电路原理图与 PCB 布局

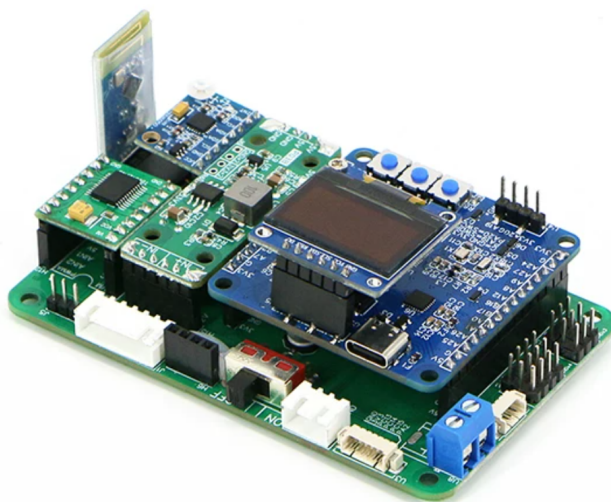
电源设计采用双路 TPS5400 降压芯片，保证各节点在满载条件下的供电稳定性；接线端子处增加滤波电容，抑制电机启停产生的电源纹波对信号链路的干扰。

3.5 自动循迹底盘控制电路

底盘子系统以 MSPM0G3507（Arm Cortex-M0+，80 MHz）为主控，连接八路模拟量灰度传感器与 MG513X 减速电机编码器。相比数字量灰度传感器，模拟量传感器可输出连续电压信号，内置自动校准功能，无需每次重启重新标定阈值，更有利于差速 Cloud PID 控制的丝滑运行。底盘控制电路原理图与实物如图3.4(a)和图3.4(b)所示。



(a) 自动循迹底盘电路原理图



(b) 自动循迹底盘电路 PCB 实物图

图 3.4 自动循迹底盘电路原理图与 PCB 实物图

底盘 PCB 集成了 MSPM0G3507 核心、电机驱动 H 桥电路、灰度传感器 ADC 接口与编码器采样电路；板载 TPS5400 为主控和传感器分别提供稳压；布局时将大电流电机驱动区与信号采集区隔开，降低电磁干扰。图3.5为 WHEELTEC C07A 底盘核心板实物图，核心芯片为 MSPM0G3507，板上集成两颗用户按键可触发运行模式切换，电机驱动与灰度传感器接口通过排针引出，便于扩展与更换。



图 3.5 WHEELTEC C07A 底盘核心板（MSPM0G3507）实物图

3.6 本章小结

本章在总体架构与器件选型的基础上，详细给出主控最小系统、IMU 采集电路、图像采集与接口、云台驱动及电源与抗干扰设计等硬件子模块的电路思路与实现要点，包含原理图、PCB 布局与接口定义。通过对供电、去耦、信号隔离与总线终端匹配的工程化处理，提升了测量可靠性与执行稳定性，为后续软件联调、整机集成与实验验证提供了可复现的硬件基础。

第4章 系统软件设计

本章基于“Linux 视觉端 + 云台主控 + 底盘主控”的分布式实现方案，围绕“感知-决策-执行”闭环，阐述系统软件架构、跨节点协同流程、控制算法、通信协议及异常处理机制。与单机式流程描述不同，本章侧重说明多控制器协同场景下的可复现实装路径，即各功能模块在 Linux 与 MCU 异构平台上的职责边界与相互之间的交互方式。

4.1 软件总体架构与流程

4.1.1 程序分层与任务划分

本课题的软件方案采用分布式分层架构，系统由三个相互独立但协同工作的计算节点构成：

(1) **Linux 视觉节点**：承担目标检测与识别、目标状态管理及视觉数据封装输出等任务；

(2) **云台主控节点 (STM32)**：负责 IMU 数据融合、双轴伺服控制、瞄准解算与云台执行闭环；

(3) **底盘主控节点 (独立单片机)**：负责路径循迹、速度闭环、转向计数与圈次任务执行。

从功能维度出发，可进一步划分为以下五层：

(1) **设备驱动层**：实现 RS485 电机通信、IMU 数据解码与串口数据接收，核心模块包括 `drv_ms4010`、`hipnuc_dec` 及 `usart_dataparsing`。

(2) **运动控制层**：由 `laser_gimbal` 模块完成双轴角度控制、速度控制、坐标指向、回零与限位管理。

(3) **视觉追踪层**：Linux 侧负责目标检测与状态跟踪，云台侧 `laser_gimbal_vision_tracker` 模块完成像素误差闭环、目标状态机切换与自动搜索逻辑。

(4) **系统集成层**：在 `main.c`、`laser_gimbal_test.c` 与 `laser_gimbal_vision_test.c` 中完成云台侧模块的初始化与运行控制，同时通过串口或总线协议与 Linux 视觉端及底盘主控端进行数据协同。

(5) **调试交互层**：通过 MSH 命令导出接口实现在线参数整定与功能验证，支撑不同场景下的快速迭代开发。

上述“节点解耦 + 功能分层”的设计将视觉计算负载、姿态控制任务与底盘实时控制任务分散至各自最适宜的平台，在降低单节点计算压力的同时提升了系统的可扩展性与工程可维护性。

4.1.2 软件主流程图

分布式系统主流程遵循“节点自检-链路建立-闭环协同”的启动顺序：

(1) **底盘节点启动**：完成电机、编码器、灰度传感器初始化，进入待命模式；

(2) **云台节点启动**：初始化 GPIO 与基础外设，调用 `laser_gimbal_system_init` 完成电机管理器建立、电机扫描、云台参数绑定、电机使能与回零；

(3) **视觉节点启动**：Linux 侧加载视觉模型与目标管理模块，基于嵌入式视觉系统框架与轻量化实现经验^[1]，建立与云台主控的数据通道并开始输出目标信息；

(4) **闭环协同运行**：云台节点持续接收视觉目标输入与 IMU 状态量，计算双轴控制量并执行瞄准；底盘节点独立执行循迹与速度闭环，同时按任务状态与上层指令完成机动。系统总体程序流程图如图4.1所示，左侧为视觉与云台控制链路，右侧为底盘循迹

控制链路，两条链路并行运行、通过接口协同。

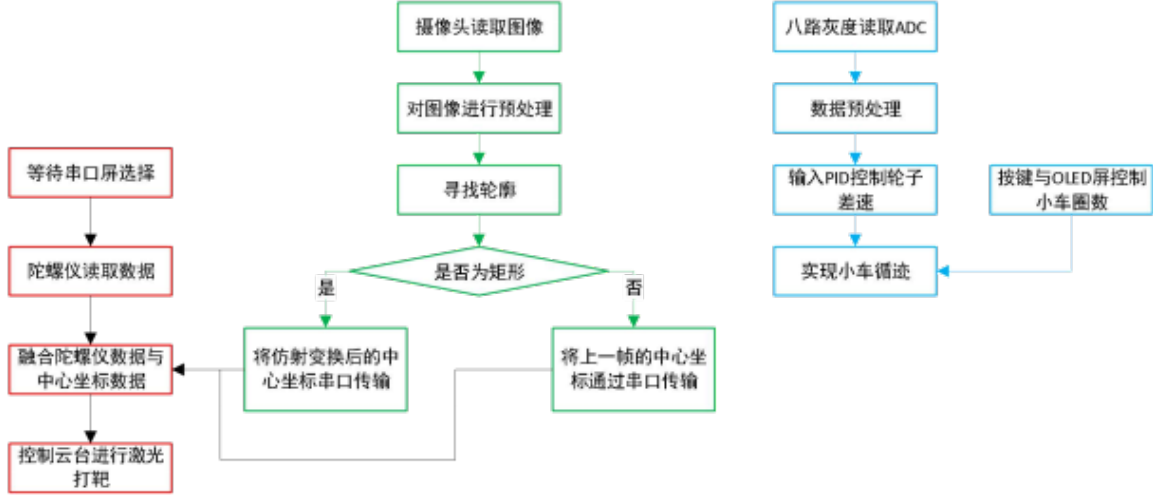


图 4.1 系统总体程序流程图

上述启动流程体现了“分布式职责清晰、局部闭环自治、跨节点协议协同”的工程原则：即使某一节点发生短时降级，其余节点仍可维持各自的最小功能闭环，从而提升了系统鲁棒性并降低了联调复杂度。

4.2 IMU 数据处理程序

4.2.1 数据采集与滤波

系统通过 `hipnuc_dec` 模块对 HiPNUC 协议进行字节流解码，先执行帧同步与 CRC 校验，再按数据包类型提取角速度、姿态角与加速度。数据解析成功后，角速度 ω 与姿态角 θ 被发布到控制路径供补偿计算使用。

在实现层面，工程采用“协议解码 + 阈值抑噪”策略：对小幅波动进行死区处理，避免将传感噪声直接放大到执行端。

4.2.2 姿态解算与角速度发布

系统在追踪线程中读取 IMU 实时状态，并构建前馈补偿项。以偏航轴为例，控制输出如式(4.1)所示，该结构参考自抗扰控制思想^[15]，在传统 PID 外环基础上叠加角速度与加速度前馈项：

$$u_{yaw} = u_{pid} + k_v \cdot \omega_{yaw} + k_a \cdot \dot{\omega}_{yaw} + k_x \cdot a_x \quad (4.1)$$

其中， u_{pid} 为视觉误差闭环输出， ω_{yaw} 为角速度项， $\dot{\omega}_{yaw}$ 为角加速度近似项， a_x 为加速度补偿项。该结构能够在平台突加速或抖动时提前修正执行量，降低纯视觉闭环的相位滞后影响^[15]。

代码4.1展示了追踪控制线程的核心逻辑：每 2 ms 一次（约 500 Hz）读取 IMU 数据计算前馈量，与视觉 APID 输出叠加后写入步进电机驱动器。

```

1 static void vision_tracker_thread_entry (void *parameter)
2 {
3     vision_tracker_t *tracker = (vision_tracker_t *)parameter;
4     float yaw_out = 0, pitch_out = 0;
5
6     while (tracker->thread_running) {
  
```

```

7     yaw_out = 0;  pitch_out = 0;
8
9     if ( tracker->enabled) {
10         if (rt_mutex_take( tracker->tracker_mutex, 10) == RT_EOK) {
11             /* 读取IMU角速度/加速度前馈量 */
12             extern hipnuc_raw_t hipnuc_raw;
13             float kv = kp_vel_yaw * hipnuc_raw.hi91.gyr[2];
14             float ka = kp_angle_yaw * hipnuc_raw.hi91.acc[0];
15
16             /* 视觉APID外环输出 */
17             vision_tracker_process_target ( tracker ,
18                                             &yaw_out, &pitch_out);
19
20             /* 叠加IMU前馈, 补偿平台抖动 */
21             yaw_out = yaw_out + kv + ka;
22
23             vision_tracker_apply_control ( tracker , yaw_out, pitch_out);
24             rt_mutex_release ( tracker->tracker_mutex);
25         }
26     }
27     rt_thread_mdelay(2); /* 控制周期~500 Hz */
28 }
29 }

```

代码 4.1: 视觉追踪线程与 IMU 前馈补偿 (laser_gimbal_vision_tracker.c)

4.3 视觉识别与跟踪程序

4.3.1 视觉线程架构与数据流

视觉端采用“三线程 + 双队列”结构, 该架构吸收了新一代实时跟踪与嵌入式跟踪思想^[7]:

(1) 读取线程 `read_thread_func`: 持续从摄像头读取帧, 并通过 `frame_queue(maxsize=1)` 仅保留最新一帧, 降低处理时延累积;

(2) 处理线程 `process_thread_func`: 完成颜色阈值分割、轮廓筛选、A4 目标判别、PnP 位姿解算与目标状态管理;

(3) 发送线程 `send_thread_func`: 从 `data_queue` 取出结果并按串口协议发送给云台主控。

该结构将“采集实时性”“算法计算量”和“通信阻塞风险”解耦, 使视觉链路在高帧率下仍具备稳定吞吐。其核心数据路径可概括为:

Camera Frame → HSV Segmentation → Contour Filter → PnP Pose → Serial Packet

读取线程核心代码如代码4.2所示, 通过 `maxsize=1` 的有界队列确保处理端始终获取到最新帧, 避免帧积压导致延迟累积。

```

1 def read_thread_func(cap):
2     """线程1: 读取视频数据, 仅保留最新一帧"""
3     while running_event.is_set() and cap.isOpened():
4         ret, frame = cap.read()
5         if not ret:
6             running_event.clear()

```

```

7         break
8     try:
9         # 清空队列，确保只保留最新的一帧
10        while not frame_queue.empty():
11            frame_queue.get_nowait()
12        frame_queue.put_nowait(frame)
13    except queue.Full:
14        pass # 并发竞争时忽略

```

代码 4.2: 视觉读取线程 (EDC_Opencv.py)

发送线程负责将处理结果格式化为 @x,y,flag,# 协议帧并通过串口发送，核心代码如代码4.3所示。

```

1 def send_thread_func(ser):
2     """线程3: 从队列取结果并按协议发送"""
3     while running_event.is_set():
4         try:
5             data = data_queue.get(timeout=1)
6             # 固定帧格式: @x,y,flag,#
7             msg = f"@{data['x']:.3 f},{data['y']:.3 f},{data['flag']:.1 f},#"
8             if ser and ser.is_open:
9                 ser.write(msg.encode('utf-8'))
10        except queue.Empty:
11            continue

```

代码 4.3: 视觉发送线程与串口协议帧 (EDC_Opencv.py)

4.3.2 目标检测与几何筛选流程

处理线程首先将图像统一到 640×480 并进行高斯滤波，随后在 HSV 空间执行 inRange 二值化和开运算去噪。阈值由 Trackbar 在线调节并可写回 trackbar_values.txt, 便于现场快速整定。

轮廓处理遵循“先粗筛、后精筛”策略：

(1) 多轮廓合并: merge_close_contours 按面积差与质心距离合并近邻轮廓，抑制边缘断裂造成的重复候选；

(2) 几何约束: 要求候选轮廓近似四边形，面积位于设定区间且宽高比满足约束；

(3) 尺寸一致性校验: find_a4_paper_contours 利用 solvePnP 和重投影尺寸估计，判断候选是否与 A4 目标物理尺寸一致。

仅当上述条件同时满足时，目标才进入后续位姿解算与下发链路。该“形状 + 尺度”双重判据可显著降低误检。几何筛选与 PnP 尺寸核查的关键代码如代码4.4所示。函数先通过 solvePnP 解算位姿，再利用重投影尺寸估计与 A4 物理尺寸比对，拒绝误匹配的候选轮廓。

```

1 def find_a4_paper_contours(contour, size_tolerance_mm=50):
2     object_points = np.array([
3         [-A4_WIDTH_MM/2, -A4_HEIGHT_MM/2, 0],
4         [ A4_WIDTH_MM/2, -A4_HEIGHT_MM/2, 0],
5         [ A4_WIDTH_MM/2,  A4_HEIGHT_MM/2, 0],
6         [-A4_WIDTH_MM/2,  A4_HEIGHT_MM/2, 0]
7     ], dtype="float32")
8     ordered_pts = order_points(contour)
9     (success, rvec, tvec) = cv2.solvePnP(object_points, ordered_pts,

```



```

10                                     CAMERA_MATRIX, DIST_COEFFS)
11     if success:
12         distance_z_mm = tvec[2][0]
13         if distance_z_mm < 100 or distance_z_mm > 2000:
14             return False
15         reprojected, _ = cv2.projectPoints(object_points, rvec, tvec,
16                                           CAMERA_MATRIX, DIST_COEFFS)
17         reprojected = reprojected.reshape(-1, 2)
18         w_px = np.linalg.norm(reprojected[1] - reprojected[0])
19         h_px = np.linalg.norm(reprojected[3] - reprojected[0])
20         est_w = (w_px * distance_z_mm) / CAMERA_MATRIX[0, 0]
21         est_h = (h_px * distance_z_mm) / CAMERA_MATRIX[1, 1]
22         if (abs(est_w - A4_WIDTH_MM) < size_tolerance_mm and
23             abs(est_h - A4_HEIGHT_MM) < size_tolerance_mm):
24             return True
25     return False

```

代码 4.4: A4 目标 PnP 尺寸筛选 (EDC_Opencv.py)

4.3.3 PnP 位姿解算与目标量生成

在 `perspective_transform_a4_pnp` 中, 系统以 A4 纸四角的三维模型点与图像角点建立对应关系, 调用 `cv2.solvePnP` 获得位姿向量 ($rvec, tvec$)。其中平移向量如式(4.2)所示:

$$tvec = [T_x, T_y, T_z]^T \quad (4.2)$$

据此可计算云台双轴角度指令, 如式(4.3)所示:

$$\theta_{pan} = \arctan\left(\frac{T_x}{T_z}\right), \quad \theta_{tilt} = \arctan\left(\frac{T_y}{T_z}\right) \quad (4.3)$$

并换算为角度输出。与此同时, 程序还返回目标中心像素 (x, y) 用于与云台侧像素闭环对接。

需要特别说明的是, 当小车在运行过程中摄像头并非始终正对矩形目标时, 目标在图像中会因透视关系产生严重的几何畸变, 此时直接取检测到的四边形几何中心将引入系统误差。为此, 本工程参考仿射变换校正策略: 利用已知 A4 纸的宽高比, 先对检测到的四边形执行透视变换, 将其校正为标准比例的矩形后取中心点, 再通过逆变换将中心坐标映射回原始图像坐标系, 从而得到消除畸变影响的精确中心位置。该处理在 PnP 解算之前执行, 可有效提升远距离或大视角偏转工况下的中心定位精度。

4.3.4 丢失重捕获与串口通信协议

工程通过 `find_target_count` 与 `lose_target_count` 实现轻量状态机:

- (1) 检测稳定阶段输出当前目标坐标;
- (2) 短时丢失阶段保持上次有效结果, 避免控制突跳;
- (3) 持续丢失阶段回退到中心搜索点 (640, 240), 并保留最近一次姿态估计作为过渡信息。

视觉端最终以固定格式发送: `@x,y,flag,#`。其中 `flag` 用于表征当前帧目标状态, 云台主控据此决定跟踪、保持或搜索策略。通过该协议, Linux 视觉节点与 STM32 云台节点形成低耦合、可扩展的数据接口。

云台侧的串口帧解析通过状态机实现, 代码4.5展示了 `Usart_Dataparsing_Handle` 中逐字节处理 `@...#` 帧的核心逻辑。

```

1 void Usart_Dataparsing_Handle(Data_Parsing_Typedef *data,
2                               uint8_t rx_data)
3 {
4     if (data->statue == 0 && rx_data == data->frame_start) {
5         memset(data->frame_string, '\0', 100);
6         data->frame_lenght = 0;
7         data->statue = 1; // 进入帧接收状态
8     } else if (data->statue == 1
9                && rx_data != data->frame_end) { // 数据段
10        data->frame_string[data->frame_lenght] = rx_data;
11        data->frame_lenght++;
12    } else if (data->statue == 1
13               && rx_data == data->frame_end) { // 帧结束
14        Usart_Dataparsing(data); // 解析并分类各字段
15        // 将解析结果送入追踪器: float[0]=x, [1]=y, [2]=confidence
16        vision_tracker_input_target (
17            &g_vision_tracker,
18            camera_data.data_buff.float_data [0],
19            camera_data.data_buff.float_data [1],
20            camera_data.data_buff.float_data [2]);
21        data->statue = 0; // 复位等待下一帧
22    }
23 }

```

代码 4.5: 串口协议帧解析状态机 (usart_dataparsing.c)

4.4 双闭环控制程序设计

4.4.1 像素误差外环 PID

外环以图像中心偏差为输入，通过 APID 控制器计算期望角速度输出。其核心目标是：

- (1) 降低稳态指向误差；
- (2) 在目标横向快速移动时维持可接受的动态超调；
- (3) 与内层补偿项协同，避免高频振荡。

通过限制 PID 输出与偏差死区，工程在可达性与稳定性之间取得了折中。

4.4.2 IMU 角速度前馈内环

内层以 IMU 角速度及其变化率作为前馈项直接叠加控制输出，对平台机动扰动进行快速抵消。与仅依赖视觉误差的策略相比，该方法对高动态工况更鲁棒，尤其在“目标仍在画面内但载体发生快速姿态变化”场景下，可显著减少跟踪偏差峰值。

此外，系统支持在线调节 vel_kp 、 ang_kp 、 acc_kp ，便于在实车/实台架条件下快速完成增益整定。

代码4.6展示了追踪线程中像素误差到 APID 控制输出的核心计算流程：先将像素坐标偏差转换为角度误差，再经死区判断与 APID 计算生成双轴控制量。

```

1 static rt_err_t vision_tracker_process_target (
2     vision_tracker_t *tracker,
3     float *out_yaw, float *out_pitch)
4 {
5     float yaw_output = 0, pitch_output = 0;
6

```



```

7  /* 像素坐标转角度误差 */
8  float yaw_error_deg, pitch_error_deg;
9  vision_tracker_pixel_to_angle_error (
10     tracker,
11     tracker->current_target .pixel_coord.x,
12     tracker->current_target .pixel_coord.y,
13     &yaw_error_deg, &pitch_error_deg);
14
15  /* 死区判断：在死区内认为已锁定 */
16  float pixel_dist =
17     powf(tracker->current_target .pixel_coord.x
18         - tracker->camera.center_x, 2) +
19     powf(tracker->current_target .pixel_coord.y
20         - tracker->camera.center_y, 2);
21  if ( pixel_dist < tracker->deadzone_pixels
22      * tracker->deadzone_pixels) {
23      tracker->state = VISION_TRACK_STATE_LOCKED;
24      vision_tracker_apply_control ( tracker , 0, 0);
25      goto FIX;
26  }
27
28  /* APID计算控制量 */
29  uint32_t now = rt_tick_get_millisecond ();
30  yaw_output = vision_pid_calculate (
31      &tracker->pid_yaw, yaw_error_deg, now);
32  pitch_output = vision_pid_calculate (
33      &tracker->pid_pitch, pitch_error_deg, now);
34  FIX:
35      *out_yaw = yaw_output;
36      *out_pitch = pitch_output;
37      return RT_EOK;
38  }

```

代码 4.6: 视觉追踪像素误差到 APID 控制输出 (laser_gimbal_vision_tracker.c)

4.5 通信与上位机控制程序

4.5.1 USB-CDC 通信协议

工程通信分为两类：

(1) **执行链路通信**：MS4010 电机通过 RS485 总线控制，底层完成命令打包、响应解析、校验和超时处理；

(2) **感知链路通信**：视觉前端通过串口发送目标像素数据，主控端解析后直接馈入追踪器。

其中，电机总线访问采用互斥锁串行化，避免多设备并发导致的帧冲突。

4.5.2 Linux 端控制界面与参数下发

上位机可通过统一协议持续下发目标与参数，实现在线闭环调参与运行监控。尽管工程当前以嵌入式侧命令调试为主，但接口定义已具备扩展到 Linux 图形化参数面板的基础条件。

4.6 系统主程序设计 with 异常处理

主控程序采用“初始化失败快速返回 + 运行期状态检查”的异常策略：

（1）初始化阶段：任一关键模块（电机管理器、电机发现、云台初始化、视觉追踪器初始化）失败即终止后续步骤并输出日志；

（2）运行阶段：通过状态机管理目标丢失、搜索、重捕获等过程，防止控制量无意义输出；

（3）并发保护：云台对象、追踪器对象与总线通信均设置互斥锁，降低资源竞争风险。

需要指出的是，部分代码中存在互斥保护被注释的情况，后续应统一临界区策略并补充高负载工况测试，以进一步提升系统稳定性。

4.7 差速底盘工程的软件实现

在本文分布式架构中，差速底盘控制被定义为与云台控制并行的独立子系统。为完善该子系统的软件实现，本文对参考目录下 WHEELTEC_C07A_IRF_CAR 工程进行了代码级梳理。该工程围绕“红外感知-模式决策-差速执行”构建了可落地的循迹与定圈速运行闭环，可作为本系统底盘运动控制模块的直接实现依据。

4.7.1 控制主循环与运行模式

工程以定时中断作为核心控制时基，Frequency=200 对应控制周期约 5 ms。在中断中根据 Run_Mode 执行不同控制分支：

（1）停机模式（Run_Mode=0）：清零控制量并关闭驱动输出；

（2）循迹模式（Run_Mode=1）：根据红外灰度结果生成 Move_X 与 Move_Z；

（3）转向模式（Run_Mode=2）：在路口阶段执行定向转弯，并在满足条件后回切循迹模式。

主循环（empty.c）负责传感器采样、归一化与状态触发，中断负责实时控制输出，这种“前台感知 + 中断闭环”的结构能够兼顾可读性和实时性。代码4.7展示了定时中断内 Run_Mode 分支的核心逻辑，包括一阶滞后滤波对偏差的平滑处理。

```

1 void TIMER_0_INST_IRQHandler(void)
2 {
3     /* 5 ms 控制周期 (Frequency = 200) */
4     if (DL_TimerG_getPendingInterrupt(TIMER_0_INST)) {
5         if (Run_Mode == 0) { /* 停机模式 */
6             Set_PWM(0, 0);
7         }
8         if (Run_Mode == 1) { /* 循迹模式 */
9             unsigned short Normal[8];
10            if (Get_Normalize_For_User(&sensor, Normal)) {
11                last_last_mid_value = last_mid_value;
12                last_mid_value      = now_mid_value;
13                now_mid_value       = gray_count_value(Normal);
14                /* 一阶滞后滤波：抑制偏差突变引起的抖动 */
15                now_mid_value = (int)(0.7f * now_mid_value
16                                     + 0.2f * last_mid_value
17                                     + 0.1f * last_last_mid_value);
18                Move_X = fflow_speed;
19                Move_Z = -(float)now_mid_value * turn_p;
20            }
21        }
22        if (Run_Mode == 2) { /* 转向模式 */

```

```

23     Move_X = fflow_speed;
24     Move_Z = turn_speed;
25     if (++turn_count >= TURN_COUNT_MAX) {
26         turn_count = 0;
27         Run_Mode = 1;      /* 回切循迹模式 */
28     }
29 }
30 Get_Target_Encoder(Move_X, Move_Z);
31 DL_TimerG_clearInterruptStatus(TIMER_0_INST,
32                               DL_TIMER_INTERRUPT_ZERO_EVENT);
33 }
34 }

```

代码 4.7: 底盘定时中断控制主循环 (control.c)

4.7.2 红外循迹与路口判别策略

该工程使用 8 路灰度传感器, 先通过 `Get_Normalize_For_User` 得到归一化数据 `Normal[8]`。随后通过两级判别实现路径跟踪:

(1) **偏差估计:** `gray_count_value()` 采用加权法计算横向偏差, 权重从左到右对称分布 (负到正), 可连续表征赛道中心偏移量;

(2) **路口检测:** `gray_is_turn()` 按平均反射强度和前部传感器触发计数判断是否到达拐点/十字区域, 并据此触发模式切换。

在循迹模式下, 工程调用 `IRDM_line_inspection()` 将多种红外组合状态映射为转向指令, 覆盖直行、小偏转、中偏转、大偏转及直角转等场景, 从而提升复杂线路下的连续通过能力。

在灰度数据的处理流程上, 工程对 8 路原始 ADC 输出进行了四级处理以提升控制丝滑度: 均值滤波, 对每路做多次采样平均, 降低电气噪声; 归一化计算, 以黑白基准值为端点, 将各路输出映射到 $[0, 1]$ 区间, 消除传感器个体差异和安装高度变化的影响; 加权偏差估计, 对 8 路归一化值按位置权重相加求得横向偏差量, 近中心传感器权重小、远端权重大, 提高边缘感知灵敏度; 一阶滞后滤波, 对最终偏差量施加低通滤波 $y_k = (1 - \alpha)y_{k-1} + \alpha u_k$, 防止偏差突变引起的云台/底盘抖动, 保证整车运行平稳。

4.7.3 固定圈速实现与圈数管理

“固定圈速”在该工程中由“恒定前向速度 + 转弯计数”协同实现:

(1) **速度基准:** `fflow_speed` 作为循迹阶段前向速度基准, 可通过按键在线增减;

(2) **转弯计数:** 每次完成一次有效拐角后 `turn_number++`;

(3) **圈数终止:** 当 `turn_number == round_number*4` 时判定完成设定圈数 (默认按每圈 4 个关键转角计)。

该实现方式不依赖全局定位系统, 工程代价低, 适合教学与竞赛场景下的闭环里程/圈次任务。

4.7.4 差速运动学与轮速闭环

在执行层, 系统先依据差速模型进行左右轮目标速度分解, 再结合编码器反馈完成速度闭环控制。左右轮目标转速分解关系如式(4.4)所示:

$$v_L = v_x - \frac{L}{2} \omega, \quad v_R = v_x + \frac{L}{2} \omega \quad (4.4)$$

其中, v_x 为前向速度指令, ω 为角速度指令, L 为轮距。工程中 `Get_Target_Encoder()` 负责该分解, `Incremental_PI_Left/Right()` 完成增量式 PI 调节, `Set_PWM()` 输出电机 PWM。

轮速反馈由 AB 相编码器中断计数获得，实现“目标-测量-校正”的实时闭环。代码4.8列出了差速分解函数与左轮增量式 PI 调速器的具体实现，PI 输出量经双端饱和限幅后写入 PWM 寄存器，防止积分饱和导致控制量溢出。

```

1  /* 差速运动学：将纵向速度/角速度分解为左右轮目标转速 */
2  void Get_Target_Encoder(float Vx, float Vz)
3  {
4      if (Vx < 0) Vz = -Vz; /* 倒退时角速度方向取反 */
5      MotorA.Target_Encoder = Vx - Vz * Wheelspacing / 2.0f;
6      MotorB.Target_Encoder = Vx + Vz * Wheelspacing / 2.0f;
7  }
8
9  /* 增量式PI调速：累积误差增量，双端限幅防积分饱和 */
10 int Incremental_PI_Left(float Encoder, float Target)
11 {
12     static float Bias, Pwm, Last_bias;
13     Bias = Target - Encoder;
14
15     float bias_i = Velocity_KI * Bias;
16     if (bias_i > 1000) bias_i = 1000; /* 积分分量限幅 */
17     if (bias_i < -1000) bias_i = -1000;
18
19     Pwm += Velocity_KP * (Bias - Last_bias) + bias_i;
20     if (Pwm > 6000) Pwm = 6000; /* 输出总量限幅 */
21     if (Pwm < -6000) Pwm = -6000;
22
23     Last_bias = Bias;
24     return (int)Pwm;
25 }

```

代码 4.8: 差速运动学分解与增量式 PI 调速 (control.c)

4.7.5 作为系统子模块的协同与可靠性优势

在本课题中，底盘子系统与云台子系统通过接口协同，而非控制耦合。WHEELTEC 工程的实现经验表明，将底盘控制链路独立设计可带来以下优势：

(1) **互不干扰**：底盘循迹与云台瞄准分别在各自控制器内闭环运行，避免因单侧任务波动造成另一侧控制性能下降；

(2) **独立运行**：底盘可在无云台参与或云台暂时降级时继续完成机动任务，云台也可在底盘速度变化时维持自身姿态控制闭环；

(3) **可靠性更高**：故障被限制在子系统内部，便于隔离、诊断与恢复，系统整体可用性优于单控制器集中式方案。

工程实现层面，“模式状态机 + 事件触发”与“运动学分解 + 编码器 PI”共同构成底盘侧基础控制框架。后续联调中，本文以标准化接口将其接入视觉与云台链路，形成“底盘机动-目标感知-指向执行”的分布式协同闭环。

4.8 本章小结

本章给出系统的软件总体架构与运行流程，明确了 Linux 视觉端、STM32 云台主控与底盘主控的职责边界及协同流程，详细介绍了视觉线程设计、像素误差外环 APID、IMU 角速度前馈内环、通信协议与异常处理策略。文中还说明了初始化顺序、互斥保护与容错机制，以保证多节点并行时的实时性与可靠性。该软件框架为后续参数整定、联

调验证与性能评估提供了系统化、可追溯的软件依据。

第 5 章 系统调试与结果分析

本章针对所设计系统开展分模块与整机两个层次的调试工作，重点验证视觉跟踪、姿态补偿与闭环控制的实际效果，并对实验结果进行分析。各项测试均以第 2 章提出的性能指标为对照基准，形成设计目标与实测结果之间的量化对比。

5.1 调试环境与测试方案

测试环境包括移动平台、固定目标（A4 纸）、云台机构、上位机及数据记录链路。测试方案采用“单模块验证-联合调试-指标评估”的渐进式流程，确保测试结果具备可解释性。测试所涉及的核心量化指标汇总于表 5.1。

表 5.1 系统性能测试指标对照

指标项	设计目标	测试方法
视觉帧率	$\geq 25 \text{ FPS}$	USB 摄像头出厂参数保证
IMU 采样频率	$\geq 100 \text{ Hz}$	IMU 官方上位机设置及出厂参数保证
稳态指向误差	$\leq 0.8 \text{ mrad}$	云台静止/底盘运动收敛后测量
摄像头标定重投影误差	次像素级	棋盘格标定重投影误差计算
底盘循迹偏差	可稳定通过标准线路	圈速偏差与脱线实验统计

5.2 图像识别与跟踪效果测试

本节验证运动扰动条件下的识别与跟踪表现。基于近期嵌入式实时跟踪与回归范式^[7]的思想，系统采用光敏留痕靶纸统计实时误差轨迹（靶纸各圆环半径相差 1cm 车距靶纸 1.5m-0.5m）。

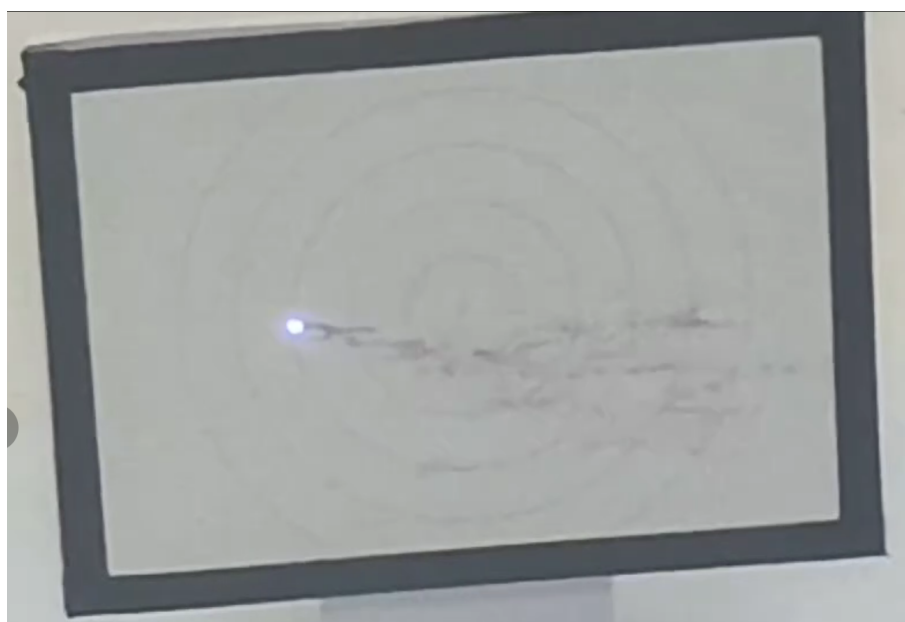


图 5.1 图像识别与跟踪效果测试

5.3 姿态采集与稳定性测试

本节评估 IMU 采样数据与稳定性效果。通过上位机时间戳统计 UART 输出频率，验证 HiPNUC IMU 在 100 Hz 配置下的实际输出稳定性：

```

5A A5 4C 00 57 C3 91 23 23 1E EB EB BE 47 93 78 00 00 18 7A 6D BF 20 38 9E 3D 45 83 B9 BE 38 5B 73 BE
78 37 6E 3D A1 D6 6C 3D 67 66 82 42 78 77 9A C1 AB AA 1C 42 5C 1F DF 42 D1 19 8F 48 94 13
[20:08:16.288] 61 41 65 C2 8D 3F 95 9D A2 8D AA 9E 52 3F E5 7E CE 3D
[20:08:16.288] 5A A5 4C 00 39 E4 91 23 23 1E D6 EB BE 47 9D 78 00 00 FD 4A 6D BF E8 16 A3 3D C3 AF BA BE
F3 A2 16 3D 6D 0E 34 BE C8 17 D5 BC 67 66 01 42 01 00 9D C1 67 66 1C 42 44 1F DF 42 1D 14 8F 48 93 17
[20:08:16.298] 61 41 FC C2 8D 3F 73 A2 A2 8D 33 9E 52 3F 39 7F CE 3D
[20:08:16.298] 5A A5 4C 00 92 68 91 23 23 1E D6 EB BE 47 A7 78 00 00 84 F6 68 BF F2 2E 98 3D 1E 5B BA BE
F5 68 C6 3E 01 DF 58 3E A6 0C C9 8D 00 00 03 42 23 22 98 C1 01 8D 1D 42 3F 28 DF 42 6F 07 8F 48 A0 08
[20:08:16.298] 61 41 5C C2 8D 3F 56 9C A2 8D 07 9E 52 3F 5A 71 CE 3D
[20:08:16.308] 5A A5 4C 00 8A 9A 91 23 23 1E D6 EB BE 47 B1 78 00 00 79 59 6C BF 38 B6 A2 3D A5 63 08 BE
5E D7 B1 8D FD 78 6E BE 08 D8 46 BE 67 66 02 42 01 00 9E C1 AB 2A 1D 42 B1 1F DF 42 9F FD 8E 48 95 9E
[20:08:16.309] 61 41 E9 C2 8D 3F A1 A1 A2 8D 08 9E 52 3F 17 6F CE 3D
[20:08:16.318] 5A A5 4C 00 F1 AD 91 23 23 1E D6 EB BE 47 B8 78 00 00 76 A5 6C BF 88 38 9F 3D 83 21 88 BE
53 6E EA BD 1A 5F 7F 3D 2D 0C 98 3E 67 E6 08 42 CD CC 99 C1 AB AA 1D 42 E4 1F DF 42 85 0A 8F 48 DA 0C
[20:08:16.319] 61 41 A3 C2 8D 3F 18 9C A2 8D AC 9E 52 3F DA 73 CE 3D
[20:08:16.328] 5A A5 4C 00 C2 E6 91 23 23 1E D6 EB BE 47 C5 78 00 00 B1 83 6C BF 98 A6 9F 3D 90 3F BA BE
CE 3D C4 BE E7 87 5A 3D A7 96 E9 3D 12 91 02 42 78 77 98 C1 56 05 1C 42 38 28 DF 42 66 08 8F 48 26 16
[20:08:16.329] 61 41 35 C2 8D 3F 8F AA A2 8D C1 9E 52 3F 9D 7A CE 3D
[20:08:16.337] 5A A5 4C 00 08 55 91 23 23 1E D6 EB BE 47 CF 78 00 00 9C 59 6C BF 05 68 A0 3D 5C 9C B9 BE
61 1F AE 3D FC 7D 12 BE 78 C8 B1 3D AB AA 03 42 23 22 98 C1 67 66 1C 42 74 1F DF 42 B1 09 8F 48 BF 17
[20:08:16.339] 61 41 E5 C2 8D 3F 08 A6 A2 8D 46 9E 52 3F 78 7A CE 3D
[20:08:16.348] 5A A5 4C 00 29 90 91 23 23 1E F5 EB BE 47 D9 78 00 00 B5 C9 68 BF 05 01 9A 3D B8 A6 BB BE
1B 71 D8 3C 44 32 8C BD F5 4D A4 BC 12 91 02 42 AB AA 9C C1 AB AA 1D 42 41 1F DF 42 39 0A 8F 48 9C 12
[20:08:16.349] 61 41 25 C3 8D 3F F7 A8 A2 8D 34 9E 52 3F A5 77 CE 3D
[20:08:16.358] 5A A5 4C 00 ED 63 91 23 23 1E F5 EB BE 47 E3 78 00 00 D8 1E 6D BF C6 08 98 3D 78 B1 BB BE
53 82 E7 BE A1 55 98 3D 3D B9 A4 8D 08 08 03 42 01 00 9E C1 01 00 1D 42 9F 1F DF 42 E6 11 8F 48 BD 1F
[20:08:16.359] 61 41 8D C2 8D 3F F6 AA A2 8D 4F 9E 52 3F 02 83 CE 3D

```

图 5.2 IMU 数据及稳定性测试

5.4 云台控制与稳态误差测试

通过典型工况测试云台执行性能，记录系统的收敛时间、超调量与稳态误差，验证双闭环控制策略对瞄准精度的改善效果。

测试工况分为两类：（1）底盘静止、目标静止——用于评估纯云台闭环的稳态误差基准（最亮光斑点）；（2）底盘匀速直线运动、目标静止——用于评估 IMU 前馈补偿对平台机动扰动的抑制效果（靶纸紫色轨迹）。

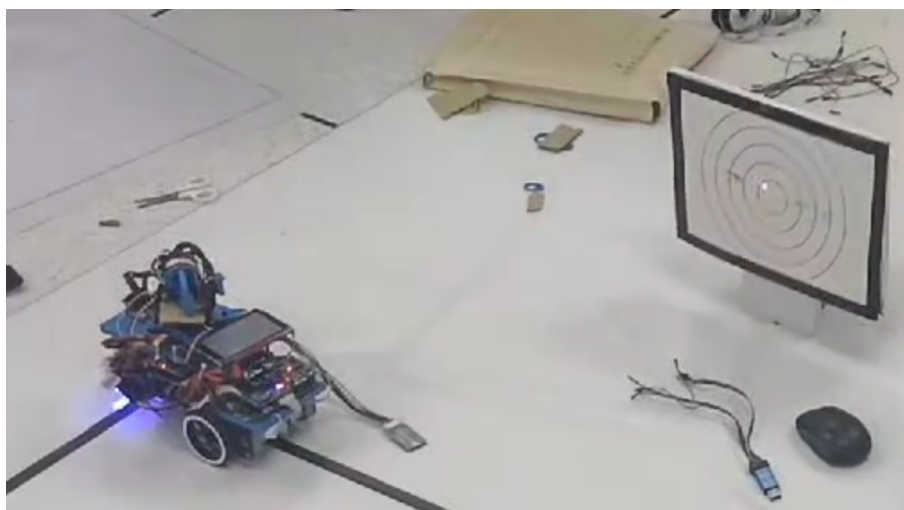


图 5.3 云台控制与稳态误差测试

5.5 本章小结

本章开展了从模块级到整机级的系统调试与实验验证，按既定性能指标对视觉帧率、IMU 采样、稳态指向误差及底盘循迹等项进行测试，记录并分析了图像识别、IMU 前馈补偿和闭环收敛的实验数据。通过对比仅视觉闭环与含 IMU 前馈的结果，量化补偿效果并总结整机联调中发现的问题与改进建议，为论文结论与后续优化提供了实验支撑与可复现的测试方法。

第6章 总结与展望

本章对全文的研究工作进行归纳与总结，提炼课题的完成情况与工程价值，并结合实验结果提出后续可进一步深化的研究方向。

6.1 主要工作与创新点

围绕“基于移动平台的固定目标识别及瞄准系统设计”这一课题，本文完成了系统总体方案制定、硬件平台搭建、软件闭环控制实现以及整机调试验证等一系列工作。

在控制架构方面，设计并实现了一种“像素误差 APID 外环 + 角速度前馈内环”的双闭环控制结构。外环以目标中心像素偏差为输入量，内环直接叠加 HiPNUC IMU 的角速度及角加速度前馈项，在平台机动或振动工况下可提前修正控制量，有效压缩纯视觉闭环的相位滞后，从而实现对静态目标的稳定指向。这一紧耦合的视觉与 IMU 双闭环架构是本课题的核心创新点。

在系统架构方面，构建了“Linux 视觉节点 + STM32H7 云台主控节点 + 独立底盘主控节点”的三节点异构分布式架构。三个节点通过标准化接口进行协同，各自在最适合的计算平台上独立闭环运行，既降低了单节点负担，也提升了系统的容错能力。该设计使得某一节点降级时不影响其他节点继续执行各自的控制任务，显著增强了整体系统的可靠性。

在工程实现方面，本文在“千元级成本、5 W 功耗、1000 g 重量”三重约束下，完成了可落地、可复现的系统工程设计。相较国外 2 万美元以上的高成本专业方案，本文为本科教学与小型无人系统研究提供了一条低成本的参考路径，对“视觉 + IMU”轻量化瞄准技术的推广具备示范意义。

在底盘控制方面，实现了基于八路灰度传感器的自主循迹、固定圈速闭环与编码器 PI 速度调节，并集成了 A* 路径规划算法，使底盘子系统具备了从“轨迹跟随”到“自主规划导航”的扩展能力，突破了固定跑道的局限。

6.2 不足与改进方向

受实验条件与开发周期所限，当前系统在多个方面仍有提升空间。首先在视觉感知层面，目标检测在强光突变、目标部分遮挡及复杂背景下的鲁棒性存在不足。当前采用的 HOG 与 HSV 相结合的方案泛化能力相对有限，其鲁棒性不及深度学习检测器。这是制约系统在各种实际光照条件下性能表现的主要因素。

其次在控制参数层面，像素闭环控制参数（PID 增益及前馈系数）目前依赖手工整定，尚不具备参数自适应能力，对不同载体机动特性的适应性有限。这意味着在面对新的平台或运动特性时，需要重新调试参数。

第三，系统的时延补偿机制仍不完善。图像传输与控制指令执行之间的时间差在高动态工况下仍会引起一定的指向滞后，这在快速机动时会逐渐累积，影响追踪精度。

最后，工程中存在部分互斥保护未完全覆盖的并发风险，需要在后续版本中补充高负载工况下的稳定性测试。同时，可考虑通过扩展多传感器融合与更高性能执行机构，在保持系统轻量化的前提下进一步提升动态工况下的瞄准精度。

6.3 后续研究工作展望

后续工作可从多个方向展开。在视觉检测升级方面，可采用轻量化深度学习模型（如 MobileNet-SSD 或 YOLO-nano）替代现有的 HSV 分割方案，以提升强光、遮挡与杂

乱背景条件下的检测鲁棒性。轻量化模型在嵌入式平台上仍可保持可接受的计算延迟，不会引入过高的系统负载。

在传感器融合层面，事件相机凭借其微秒级时间分辨率与低运动模糊特性，是高动态平台瞄准场景中极具潜力的传感器升级路径。未来可考虑以事件相机替换或补充现有 USB 摄像头，完成视觉链路的升级。

在自适应控制方面，建立基于平台运动状态的在线参数自整定机制具有重要的工程意义。与此同时，引入预测性时延补偿可进一步压缩高动态工况下的瞄准滞后，提升系统在实际运动场景中的追踪性能。

此外，将当前系统的控制框架封装为标准化的软件模块，使其能够向不同嵌入式平台迁移，可显著降低后续系统集成的成本，提升技术的可复用性，对于推动相关技术在多个应用场景中的落地具有重要价值。

参考文献

- [1] HUTCHINSON S, HAGER G D, CORKE P I. A Tutorial on Visual Servo Control[J/OL]. IEEE Transactions on Robotics and Automation, 1996, 12(5): 651 – 670.
<http://dx.doi.org/10.1109/70.538972>.
- [2] 马腾飞, 张洁, 贾培发. 视觉伺服控制综述与发展展望 [J]. 自动化学报, 2017, 43(10): 1673 – 1687.
- [3] MOOSMANN J, GIORDANO M, VOGT C, et al. TinyissimoYOLO: A Quantized, Low-Memory Footprint, TinyML Object Detection Network for Low Power Microcontrollers[J/OL]. arXiv preprint arXiv:2306.00001, 2023.
<https://arxiv.org/abs/2306.00001>.
- [4] BOYLE L, MOOSMANN J, BAUMANN N, et al. DSORT-MCU: Detecting Small Objects in Real-Time on Microcontroller Units[J/OL]. arXiv preprint arXiv:2410.16769, 2024.
<https://arxiv.org/abs/2410.16769>.
- [5] HOLLARD L, MOHIMONT L, GAVEAU N, et al. LeYOLO, New Embedded Architecture for Object Detection[J/OL]. arXiv preprint arXiv:2406.14239, 2025.
<https://arxiv.org/abs/2406.14239>.
- [6] YAO H, HE F, HAO N, et al. EdgePoint2: Compact Descriptors for Superior Efficiency and Accuracy[J/OL]. arXiv preprint arXiv:2504.17280, 2025.
<https://arxiv.org/abs/2504.17280>.
- [7] ZHANG Y, LIANG C, GAO J, et al. Temporal Correlation Meets Embedding: Towards a 2nd Generation of JDE-based Real-Time Multi-Object Tracking[J/OL]. arXiv preprint arXiv:2407.14086, 2024.
<https://arxiv.org/abs/2407.14086>.
- [8] WANG A, LIU L, CHEN H, et al. YOLOE: Real-Time Seeing Anything[J/OL]. arXiv preprint arXiv:2503.07465, 2025.
<https://arxiv.org/abs/2503.07465>.
- [9] XIAO Y, XU T, XIN Y, et al. FBRT-YOLO: Faster and Better for Real-Time Aerial Image Detection[J/OL]. arXiv preprint arXiv:2504.20670, 2025.
<https://arxiv.org/abs/2504.20670>.
- [10] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[C/OL] //. 2016.
http://dx.doi.org/10.1007/978-3-319-46466-4_50.
- [11] 刘昊, 王晓慧, 李培国. 嵌入式计算机视觉中的轻量化深度学习方法 [J]. 计算机研究与发展, 2018, 55(10): 2213 – 2229.

- [12] HAN S, MAO H, DALLY W J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding[C] // International Conference on Learning Representations (ICLR). 2016 : 1 – 13.
- [13] SANDLER M, HOWARD A, ZHU M, et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks[C/OL] // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018 : 4510 – 4520.
<http://dx.doi.org/10.1109/CVPR.2018.00474>.
- [14] ZHANG Z. A Flexible New Technique for Camera Calibration[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000, 22(11): 1330 – 1334.
<http://dx.doi.org/10.1109/34.888718>.
- [15] HAN J. From PID to Active Disturbance Rejection Control[J/OL]. IEEE Transactions on Industrial Electronics, 2009, 56(3): 900 – 906.
<http://dx.doi.org/10.1109/TIE.2009.2017071>.

致谢

本文的完成，离不开众多师长、同学与亲友的关心与支持，在此谨致以最诚挚的谢意。

衷心感谢导师蔡军教授。蔡老师学识渊博、治学严谨，自课题立项至论文定稿的全过程中，始终给予悉心的指导与鼓励。在研究方法的选取、实验方案的设计以及论文撰写的各个环节，蔡老师都提出了宝贵的意见和建议，使本文的研究工作得以顺利推进。蔡老师严谨求实的学术态度和一丝不苟的工作精神，将对我今后的学习与工作产生深远影响。

感谢实验室各位同学在日常学习与科研工作中给予的热情帮助与支持，共同的讨论与协作使本课题的研究工作更加充实、高效。感谢同组同学在硬件调试、代码联调和实验测试阶段提供的密切配合，正是大家的协力协作使系统得以顺利集成并成功功能验证。

感谢重庆邮电大学自动化/工业互联网学院各位任课教师，在过去数年中对我在专业知识与综合能力方面的悉心培养，为本课题研究奠定了坚实的理论与工程基础。

最后，对使用的各大 AI 模型表示感谢。它们渊博的知识是我完成学业最坚强的后盾。

附录 A 科技写作中非学术型低级错误的主要表现

本附录主要针对学位论文写作或中文科技论文写作，供重庆邮电大学学位论文查非工作参考。未尽事宜，可参考重庆邮电大学论文写作要求、重庆邮电大学学报编辑部等国内期刊社、出版社的通用出版规定。

推荐阅读《科学出版社作者编辑手册》、《科学道德与学风建设宣传参考大纲（试用本）》等写作指导性书籍或资料，可了解更多、更详尽的通用写作出版规范。

（注：对于一些不宜放入正文中，但作为毕业设计（论文）又不可缺的组成部分或具有重要参考价值的内容，可编入毕业设计（论文）的附录中，例如，公式的推演、源程序代码、附图等内容。附录的内容为备选项目，作者可根据内容的需要决定附录的项目数，用附录 A、附录 B 方式编号。附录的篇幅不宜太多。附录与主体部分一起编制页码。若附录部分有手工制作或复印件，手工制作或复印件部分要装订在内但不计页码。附录的文字按照正文格式进行排版。）

（附：科技写作中非学术型低级错误的主要表现见 moban 文件夹下）

代码效果演示：

```
1 # 这是一段代码测试
2 def CQUPT():
3     print("Hello,CQUPT")
4     print("学长学长，给咱们讲讲3G芯片的故事呗。")
5     print("这是世界上第一枚0.13微米工艺的TD-SCDMA 3G手机基带芯片。")
6     + "它的诞生，标志着我国3G通信核心芯片等关键..."
7 CQUPT()
```


附录 B 英文翻译

指导教师制定与专业相关的外文文献内容，由学生独立翻译成中文，其外文文献内容翻译成中文后的内容不得少于 3000 字符。译文和原文附于附录部分，按照正文格式进行排版。若原文没有电子版只是原版的复印件，复印件装订在内可不计页码。